



Generación y Optimización de Código (80 horas)



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

Generación de código



➤ Generación de código en distintos lenguajes con IA.

- I. *Lenguajes soportados por IA*
- II. *Estrategias para generar código efectivamente*
- III. *Técnicas avanzadas de generación*



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

Objetivos del Módulo

- Dominar la generación de código en múltiples lenguajes con IA
- Aplicar técnicas avanzadas de refactorización asistida
- Crear funciones y algoritmos complejos con IA
- Optimizar código para rendimiento y mantenibilidad
- Implementar un workflow profesional de colaboración humano-IA

Dominar la generación de código en múltiples lenguajes con IA

Aprenderemos a utilizar herramientas como Copilot, ChatGPT o CodeWhisperer para generar código en distintos lenguajes, entendiendo:

- Cómo adaptar prompts según el lenguaje
- Qué lenguajes tienen mejor soporte
- Cómo guiar a la IA para obtener código idiomático
- Cómo evitar errores comunes (dependencias inexistentes, versiones incorrectas, etc.)

Dominar la generación de código en múltiples lenguajes con IA

✓ Ejemplos"

- "Genera un endpoint en FastAPI con validación Pydantic."
- "Crea un componente React con TypeScript y hooks."
- "Implementa una clase en Java con Spring Boot y logging."

Aplicar técnicas avanzadas de refactorización asistida

Aprenderemos a usar IA para:

- Detectar *code smells*
- Simplificar condicionales
- Modularizar funciones grandes
- Aplicar patrones de diseño
- Modernizar código legacy
- Reducir complejidad ciclomática

Aplicar técnicas avanzadas de refactorización asistida

✓ Ejemplos

- "Refactoriza este código usando el patrón Strategy."
- "Simplifica esta función eliminando anidación innecesaria."
- "Convierte este código jQuery a React Hooks."

Crear funciones y algoritmos complejos con IA

Aprenderemos a generar:

- Algoritmos de búsqueda (A*, BFS, DFS)
- Algoritmos de ordenamiento (QuickSort, MergeSort)
- Estructuras de datos (árboles, grafos, tries)
- Parsers, sistemas de permisos, lógica de negocio compleja

La IA acelera la implementación, pero debemos validar:

Complejidad temporal

Correctitud

Casos límite

Crear funciones y algoritmos complejos con IA

✓ Ejemplos

- "Implementa un Trie en TypeScript para autocompletado."
- "Crea QuickSort optimizado para arrays parcialmente ordenados."

Optimizar código para rendimiento y mantenibilidad

Aprenderemos a usar IA para:

Detectar cuellos de botella

Mejorar complejidad algorítmica

Reducir uso de memoria

Reescribir funciones ineficientes

Mejorar legibilidad y mantenibilidad

Optimizar código para rendimiento y mantenibilidad

✓ Ejemplos

"Optimiza este algoritmo $O(n^2)$ a $O(n)$."

"Reduce el uso de memoria en este procesamiento de listas."

Implementar un workflow profesional de colaboración humano-IA

Aprenderemos a integrar IA en un flujo de trabajo real:

Diseño → IA genera scaffolding

Implementación → humano + IA

Refactorización → IA propone, humano decide

Testing → IA genera tests, humano valida

Documentación → IA redacta borradores

Implementar un workflow profesional de colaboración humano-IA

✓ Ejemplos

"Genera la estructura base de una API y yo la completo."

"Crea tests unitarios y yo añado casos edge."

Generación de Código en Distintos Lenguajes

Las herramientas modernas de IA soportan más de 40 lenguajes, pero su rendimiento varía según:

- Cantidad de documentación pública
- Popularidad del lenguaje
- Calidad del ecosistema
- Uso en repositorios públicos

Generación de Código en Distintos Lenguajes

Generación de código



Los lenguajes con mejor soporte son:

- ◆ Python
 - Excelente soporte
 - Ideal para backend, scripting, data science
 - La IA genera código muy idiomático
- ◆ JavaScript / TypeScript
 - Perfecto para frontend y Node.js
 - Muy buen soporte en React, Express, Next.js

Generación de Código en Distintos Lenguajes

Los lenguajes con mejor soporte son:

- ◆ Java
 - Muy sólido en Spring Boot
 - Buenas sugerencias de patrones y anotaciones
- ◆ C#(C Sharp)
 - Ideal para .NET y Unity
 - Buen soporte para clases, interfaces y patrones

Generación de Código en Distintos Lenguajes

Los lenguajes con mejor soporte son:

- ◆ C/C++

Soporte aceptable

Útil para estructuras de datos y algoritmos

- ◆ Go

Muy buen soporte para microservicios

La IA genera código limpio y conciso

Generación de Código en Distintos Lenguajes

Los lenguajes con mejor soporte son:

- ◆ Rust
 - Soporte creciente
 - La IA ayuda mucho con borrow checker y ownership
- ◆ PHP
 - Soporte estable
 - Útil para Laravel y proyectos web tradicionales

LA IA funciona especialmente bien en lenguajes con mucha documentación pública y repositorios abiertos.

Python, JavaScript y Java suelen ofrecer los mejores resultados.



La calidad del código generado por IA depende directamente de cómo se formula el prompt.

Estas estrategias permiten obtener código más preciso, seguro, actualizado y adaptado al proyecto.

1. Especificación clara

Qué significa

La IA necesita instrucciones concretas: qué debe hacer, cómo debe hacerlo y bajo qué condiciones.

Qué incluir

- ☐ Lenguaje
- ☐ Función exacta
- ☐ Requisitos técnicos
- ☐ Formato de salida
- ☐ Validaciones necesarias

✓ Ejemplo

"Crea una función en Python que valide emails usando regex.

Debe devolver True/False y manejar entradas nulas."

✓ Por qué funciona

Reduce ambigüedad y evita que la IA invente detalles.

2. Contexto del proyecto

Qué significa

La IA genera mejor código cuando sabe dónde se va a usar.

Qué incluir

- ☐ Framework
- ☐ Versión
- ☐ Arquitectura
- ☐ Librerías ya instaladas
- ☐ Estilo del proyecto

✓ Ejemplo

"Usando React 18 y TypeScript, crea un hook(función) personalizado para fetch de datos(HTTP) con estado de loading(petición) y error."

✓ Por qué funciona

Evita código incompatible o desactualizado.

■ 3. Ejemplos de entrada y salida

✓ Qué significa

Mostrar a la IA cómo debe comportarse la función.

✓ Qué incluir

- ☐ Inputs reales
- ☐ Outputs esperados
- ☐ Casos límite

✓ Ejemplo

"Entrada: [1,2,3] → Salida: 6.

Entrada: [] → Salida: 0."

✓ Por qué funciona

La IA no adivina. Observa de tus ejemplos y busca la regla que los conecta. Si los ejemplos son claros y coherentes, la IA aprende exactamente la transformación que quieres

4. Restricciones técnicas

Qué significa

Limitar lo que la IA puede usar o cómo debe implementarlo.

Qué incluir

- ☐ Complejidad deseada
- ☐ Memoria
- ☐ Dependencias permitidas
- ☐ Estándares de seguridad

✓ Ejemplo

"Implementa búsqueda binaria en Go sin usar librerías externas.

Con un requisito de calidad = Complejidad $O(\log n)$."

✓ Por qué funciona

Porque no da un resultado esperado: un algoritmo correcto, eficiente y escrito solo con la librería estándar.

Este tipo de instrucción es muy precisa: no deja espacio para interpretaciones ambiguas.

■ 5. Estilo de código

✓ Qué significa

Indicar cómo debe verse el código final.

✓ Qué incluir

- ☐ Convenciones (PEP8, SOLID, Clean Code...)
- ☐ Nombres descriptivos
- ☐ Comentarios o docstrings
- ☐ Estructura de carpetas

✓ Ejemplo

“Usando principios SOLID, crea una clase UserService en Java con inyección de dependencias y manejo de excepciones.”

✓ Por qué funciona

El código generado encaja con el resto del proyecto, y la IA entiende que hemos utilizado try/catch y aplicamos el **Single Responsibility**, **Dependency Inversion**, etc.

6. Iteración y refinamiento

Qué significa

No pedir todo de golpe.

La IA funciona mejor por pasos.

Qué debe incluir

- ☐ Objetivos pequeños y claros
- ☐ Validación entre pasos
- ☐ Correcciones o ajustes

✓ Ejemplo de flujo

- "Crea la estructura base de una API REST en Express."
- "Añade autenticación JWT."
- "Implementa manejo de errores centralizado."
- "Optimiza consultas con caché Redis."

✓ Por qué funciona

Evita errores y genera código más limpio.



■ 7. Validación inmediata

✓ Qué significa

Pedir a la IA que revise su propio código.

✓ Qué debe incluir

- ☐ Revisión funcional
- ☐ Revisión de errores comunes
- ☐ Revisión de estilo, eficiencia y seguridad

✓ Ejemplo

“Revisa el código anterior y señala posibles errores, dependencias faltantes o mejoras de seguridad.”

✓ Por qué funciona

Reduce errores sutiles y mejora la calidad final, ahorra tiempo de depuración y refuerza el aprendizaje al mostrar buenas prácticas.

8. Pedir alternativas

Qué significa

Solicitar varias versiones del mismo código..

Qué debe incluir

- ☐ Variación real entre opciones
- ☐ Límites o requisitos comunes
- ☐ Explicación breve de cada alternativa

✓ Ejemplo

“Dame tres implementaciones distintas: una simple, una optimizada y una usando programación funcional.”

✓ Por qué funciona

- Permite comparar enfoques y elegir el mejor.
- Ayuda a entender diferentes estilos de programación.
- Evita quedarse con la primera solución generada.
- Fomenta pensamiento crítico y buenas prácticas.

■ 9. Controlar el nivel de detalle

✓ Qué significa

Indicar si quieres código completo o solo la estructura.

✓ Qué debe incluir

- ☐ Alcance exacto
- ☐ Profundidad y formato
- ☐ Exclusiones claras

✓ Ejemplo

“Dame solo la estructura de carpetas y archivos, sin implementar la lógica.”

✓ Por qué funciona

- Evita que la IA genere demasiado código innecesario.
- Reduce ruido y facilita el enfoque en la parte importante.
- Ahorra tiempo de lectura y revisión.
- Permite construir el proyecto paso a paso.

10. Pedir explicaciones junto al código

✓ Qué significa

Solicitar que la IA genere código acompañado de explicaciones claras, paso a paso, indicando qué hace cada parte y por qué se ha escrito así..

✓ Qué debe incluir

- ☐ Explicación línea por línea o bloque por bloque
- ☐ Justificación técnica
- ☐ Relación entre código y teoría

✓ Ejemplo

"Genera el código y explica cada paso como si fuera para un alumno de FP."

✓ Por qué funciona

- Facilita la comprensión y la enseñanza.
- Ayuda a entender cómo piensa un programador profesional.
- Reduce la curva de aprendizaje en tecnologías nuevas.
- Permite detectar errores conceptuales antes de escribir más



Ejemplos prácticos

Ejemplo 1 — Prompt malo

"Hazme una API."

Resultado: ¿?

Ejemplo 2 — Prompt bueno

"Crea una API REST en FastAPI con endpoints CRUD para usuarios, validación Pydantic, manejo de errores y documentación automática con OpenAPI."

Resultado:¿?

Las técnicas avanzadas permiten pasar de “la IA me genera código” a “la IA me ayuda a construir sistemas completos, optimizados y mantenibles”.

Este apartado vamos a trabajar con la IA como un co-desarrollador real.

1. Generación Iterativa (4 pasos)

La IA funciona mejor cuando se le pide el código por etapas, no todo de golpe.

- ◆ Paso 1 — Estructura base
- ◆ Paso 2 — Añadir funcionalidad
- ◆ Paso 3 — Manejo de errores
- ◆ Paso 4 — Optimización

◆ Paso 1 — Estructura base

Se solicita únicamente el esqueleto del proyecto:

- ☐ Carpetas
- ☐ Archivos
- ☐ Rutas vacías
- ☐ Clases sin lógica

Ejemplo:

"Crea la estructura base de una API RESTful(GET, POST, PUT, DELETE) en Node.js con Express, con carpetas para routes, controllers y services."

◆ Paso 2 — Añadir funcionalidad

Una vez creada la estructura, se añaden piezas específicas:

- ☐ Endpoints
- ☐ Validaciones
- ☐ Modelos
- ☐ Servicios

Ejemplo:

"Añade autenticación JWT(JSON Web Tokens) a la API anterior, con rutas /login y /register."

◆ Paso 3 — Manejo de errores

Se pide a la IA que implemente:

- ❑ Middlewares(try/catch)
- ❑ Excepciones personalizadas(ValidationError, AuthError)
- ❑ Logs(Winston, Log4js)
- ❑ Respuestas estándar

```
{  
  "status": "error",  
  "code": 400,  
  "message": "Datos inválidos",  
  "details": {...}  
}
```

Ejemplo:

"Implementa un middleware de manejo de errores centralizado con códigos HTTP adecuados."

◆ Paso 4 — Optimización

Finalmente, se optimiza:

- ☐ Caché
- ☐ Consultas
- ☐ Complejidad
- ☐ Seguridad

Ejemplo:

"Añade caché con Redis para las consultas GET más frecuentes."

■ 2. Patrones de Prompts por Lenguaje

Cada lenguaje tiene convenciones y estilos propios.

La IA genera mejor código cuando se le pide siguiendo esos patrones.

- ◆ Python
- ◆ React / TypeScript
- ◆ Java

◆ Python

- ❑ `type hints` para claridad y validación
- ❑ `docstrings` para documentación interna
- ❑ `Pydantic` para validación de datos
- ❑ Estructura clara: clases, funciones, módulos separados
- ❑ Estilo PEP8: nombres en `snake_case`, indentación de 4 espacios

Prompt ideal

"Crea una clase Python con type hints, docstrings y validación de datos usando Pydantic."

 Resultado esperado, la IA genera algo como:

```
from pydantic import BaseModel

class User(BaseModel):
    id: int
    name: str
    email: str

    def greet(self) -> str:
        """Devuelve un saludo personalizado."""
        return f"Hola, {self.name}!"
```

◆ JavaScript / TypeScript (frontend y backend)

- ☐ React 18
- ☐ Hooks
- ☐ Interfaces TS
- ☐ Node.js + Express
- ☐ Next.js

🧩 Prompt ideal

"Genera un componente React con TypeScript, props tipadas, validación de formularios y manejo de estado con useReducer."

◆ Java (Spring Boot)

- ☐ Anotaciones
- ☐ Logging
- ☐ Excepciones personalizadas
- ☐ Inyección de dependencias

🧩 Prompt ideal

“Implementa un servicio en Spring Boot con @Service, manejo de excepciones, logs con SLF4J y pruebas con JUnit.

■ 3. Comentarios que generan código automáticamente
Las herramientas IA pueden generar código a partir de comentarios.

Ejemplos:

// TODO: Función que calcule el promedio de una lista

/* API endpoint para obtener usuarios activos */

/// <summary>Valida formato de email</summary>

Clase que maneja conexión a PostgreSQL

Así se aprende a documentar y a generar código limpio a la vez.

4. Control de versiones y compatibilidad

La IA puede generar código obsoleto si no se especifica versión.

Ejemplos:

- ☐ React 17 vs React 18
- ☐ Node 14 vs Node 20
- ☐ Python 3.7 vs Python 3.12
- ☐ Spring Boot 2 vs Spring Boot 3

Prompt recomendado:

“Usa React 18 y TypeScript 5.0. No uses clases, solo componentes funcionales.”

4. Control de versiones y compatibilidad

La IA puede generar código obsoleto si no se especifica versión.

Ejemplos:

- ☐ React 17 vs React 18
- ☐ Node 14 vs Node 20
- ☐ Python 3.7 vs Python 3.12
- ☐ Spring Boot 2 vs Spring Boot 3

Prompt recomendado:

“Usa React 18 y TypeScript 5.0. No uses clases, solo componentes funcionales.”

5. Generación guiada por restricciones

La IA responde mejor cuando se le imponen límites:

- ☐ Complejidad
- ☐ Memoria
- ☐ Seguridad
- ☐ Librerías permitidas

Ejemplo:

Implementa QuickSort en Go con complejidad $O(n \log n)$, sin librerías externas."

6. Generación con estilo de código

La IA puede adaptarse al estilo del proyecto:

SOLID

Clean Code

Arquitectura hexagonal

Nombres descriptivos

Comentarios obligatorios

Ejemplo:

“Genera un controlador en Java siguiendo arquitectura hexagonal y principios SOLID.”

■ 7. Validación automática del código generado

Se puede pedir a la IA que revise su propio código:

Ejemplo:

“Revisa el código anterior y señala posibles errores, dependencias faltantes o mejoras de seguridad.”

8. Generación de alternativas

Pedir varias versiones permite comparar:

- ☐ Simple
- ☐ Optimizada
- ☐ Funcional
- ☐ Orientada a objetos

Ejemplo:

“Dame tres implementaciones distintas de esta función: una simple, una optimizada y una usando programación funcional.”

9. Generación modular

Pedir módulos separados:

- ☐ Modelos
- ☐ Servicios
- ☐ Controladores
- ☐ Tests
- ☐ Documentación

Ejemplo:

“Genera solo el archivo **user.service.ts** con las funciones **CRUD**.”



Gracias.



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN



